

## Lab No 10:

### Programs using different CUDA Device memory types and synchronization

**Synchronization:** CUDA allows threads in the same block to coordinate their activities using a barrier synchronization function, `__syncthreads()`. When a kernel function calls `__syncthreads()`, the thread that executes the function call will be held at the calling location until every thread in the block reaches the location. This ensures that all threads in a block have completed a phase of their execution of the kernel before any moves on to the next phase.

**Shared Variables:** Accessing shared memory is extremely fast and highly parallel. If a variable declaration is preceded by the keyword `__shared__`, it declares a shared variable in CUDA. Such declarations typically reside within a kernel function or a device function. The scope of a shared variable is within a thread block means all threads in a block see the same version of a shared variable. The lifetime of a shared variable is within the duration of the kernel. Shared variables are an efficient means for threads within a block to collaborate with each other. Shared memory is fast but it is small. A common strategy is partition the data into subsets called **tiles** so that each tile fits into the shared memory.

**Constant Variables:** If a variable declaration is preceded by the keyword `__constant__`, it declares a constant variable in CUDA. Declaration of constant variables must be outside any function body. The scope of a constant variable is all grids, meaning that all threads in all grids see the same version of a constant variable. The lifetime of a constant variable is the entire application execution. Constant variables are stored in the global memory but are cached for efficient access. With appropriate access patterns, accessing constant memory is extremely fast and parallel. Currently, the total size of constant variables in an application is limited at 65,536 bytes. One may need to break up the input data volume to fit within this limitation.

**Device Variables:** A variable whose declaration is preceded only by the keyword `__device__` is a global variable and will be placed in global memory. Accesses to a global variable are slow. However, global variables are visible to all threads of all kernels. Their contents also persist through the entire execution. Thus, global variables can be used as a means for threads to collaborate across blocks. Global variables are often used to pass information from one kernel invocation to another kernel invocation.

Solved Exercise:

Write a program in CUDA to perform tiled matrix multiplication using 2D Grid and 2D Block

```
//Matrix multiplication of 4x4 matrix
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <cuda_runtime.h>
#define BLOCK_WIDTH 2
#define TILE_WIDTH 2
#define WIDTH 4

__global__ void MatMulElementThreadShared(int *a, int *b, int *c) {
    __shared__ int MDs[TILE_WIDTH][TILE_WIDTH];
    __shared__ int NDs[TILE_WIDTH][TILE_WIDTH];
    int m;
    int bx=blockIdx.x; int by=blockIdx.y;
    int tx=threadIdx.x; int ty=threadIdx.y;

    int Row=by*TILE_WIDTH + ty;
    int Col= bx*TILE_WIDTH + tx;

    int Pvalue=0;
    for(m=0; m<WIDTH/TILE_WIDTH; m++)
    {
        MDs[ty][tx]=a[Row*WIDTH+m*TILE_WIDTH+tx];
        NDs[ty][tx]=b[(m*TILE_WIDTH+ty)*WIDTH+Col];

        __syncthreads();

        for (int k = 0; k < TILE_WIDTH; k++)
        {
            Pvalue += MDs[ty][k]*NDs[k][tx];
        }
        __syncthreads();
    }
    c[Row*WIDTH+Col] = Pvalue;
}
```

```

int main() {
    int *matA, *matB, *matProd;
    int *da, *db, *dc;

    printf("\n== Enter elements of Matrix A (4x4) ==\n");

    matA = (int*)malloc(sizeof(int) * WIDTH * WIDTH);
    for(int i = 0; i < WIDTH * WIDTH; i++)
    {
        scanf("%d", &matA[i]);
    }

    printf("\n== Enter elements of Matrix B (4x4) ==\n");
    matB = (int*)malloc(sizeof(int) * WIDTH * WIDTH);
    for(int i = 0; i < WIDTH * WIDTH; i++)
    {
        scanf("%d", &matB[i]);
    }
    matProd = (int*)malloc(sizeof(int) * WIDTH * WIDTH);

    cudaMalloc((void **) &da, sizeof(int) * WIDTH * WIDTH);
    cudaMalloc((void **) &db, sizeof(int) * WIDTH * WIDTH);
    cudaMalloc((void **) &dc, sizeof(int) * WIDTH * WIDTH);

    cudaMemcpy(da, matA, sizeof(int) * WIDTH * WIDTH, cudaMemcpyHostToDevice);
    cudaMemcpy(db, matB, sizeof(int) * WIDTH * WIDTH, cudaMemcpyHostToDevice);
    int NumBlocks = WIDTH / BLOCK_WIDTH;
    dim3 grid_conf (NumBlocks, NumBlocks);
    dim3 block_conf (BLOCK_WIDTH, BLOCK_WIDTH);

    MatMulElementThreadShared<<<grid_conf, block_conf>>>(da, db, dc);

    cudaMemcpy(matProd, dc, sizeof(int) * WIDTH * WIDTH, cudaMemcpyDeviceToHost);
    printf("\n==Result of Addition==\n");
    printf("-----\n");
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            printf("%6d ", matProd[i * n + j]);
        }
    }
}

```

```
        printf("\n");
    }
    cudaFree(da);
    cudaFree(db);
    cudaFree(dc);
    free(matA);
    free(matB);
    free(matProd);
    return 0;
}
```

#### Lab Exercises:

1. Implement a program in CUDA to improve the performance of 1D parallel convolution using constant Memory.
2. Implement a program in CUDA to perform tiled 1D convolution operation on the input array N of size *width* using the mask array, M of size *mask\_width*, to produce the resultant array P of size *width*.
3. Implement a program in CUDA to perform matrix multiplication using 2D Grid of 2D Block.

#### Additional Exercise:

1. Implement a program in CUDA which displays a shopping mall item menu with its price. The N number of friends are allowed to purchase as many items as possible they want. Calculate the total purchase done by N friends.

